

Dynamic Segment Size Selection in HTTP Based Adaptive Video Streaming

Luca Bedogni*, Marco Di Felice*, Luciano Bononi*

*Department of Computer Science and Engineering, University of Bologna, Italy

*Email: {lbedogni, difelice, bononi}@cs.unibo.it,

Abstract—Video streaming takes the lion's share of network bandwidth, with a trend that will likely increase in the future. In the recent years, dynamic adaptive streaming has been developed to offer a smooth video stream with variable quality, depending on the performance of the network connection. At present, several content providers like Netflix, YouTube and Hulu, to name a few, already offer videos that can be streamed with a dynamic video quality. In this work we study the tradeoff of the video length to be downloaded, called segment size, and show the differences in terms of playback quality and reduced buffer outages. We then propose an algorithm able to exploit the network conditions, and adaptively select the video segment size to be downloaded. We analyze the benefits of our proposal in terms of increased playback quality and reduced buffer outages against classic fixed segment size solutions. Our result show that dynamically adapting the segment size can reduce buffer outages, while also increasing the quality streamed.

I. INTRODUCTION

In recent years we have witnessed a constant and relentless increase in the amount of data that has been published or requested on the Internet. According to [2], video streaming accounts for the lion's share of the bandwidth, using an impressive 55 % of the mobile data bandwidth. The 2020 forecast puts mobile video streaming at 75 % of the total bandwidth used. At the same time, the average network speed is increasing [2], mostly thanks to LTE and to open WIFI deployments. However, mobile networks can suffer from huge data rate variations when moving, mostly due to shadowing effects. To cope with a dynamic environment, several adaptive streaming standards have been proposed, like Microsoft SmoothStreaming, Adobe ADS, Apple HLS, and MPEG-DASH [13]. In this paper we focus on the latter, but we note that our proposal is general and adaptable to other standards. Regardless of the standard, typically video chunks are encoded at different qualities, and thus different sizes, and the client will download the most appropriate one according to its bandwidth.

Adaptive video streaming is used by many popular video streaming providers, such as YouTube, Hulu, and Netflix. The main idea is to monitor the network conditions, and adapt to the data rate offered by the communication technology. This is to prevent possible pauses of the video, which are accounted as one of the worst problems that can happen during a video playback [4], as users tend to react more positively when the quality is reduced, rather than having the video to stop [3].

Before streaming a video, the MPEG-DASH client should firstly download its manifest, consisting of several meta in-

formation, including the different video quality at which the video can be played, the average bitrate, and the video length. Often, clients implicitly select a video segment size (SS), that is, the length of the file they will download to play a portion of the video. Others might have the SS built-in, hence the selection has been made either by the developer of the client or by the content provider. This selection is made at the beginning, and will typically remain the same for the duration of the video playback. However, as we show in this work, the SS selection has a direct impact on the quality of experience for the end users. A difference between the different SS is the file size to be downloaded, according to the defined Group of Pictures (GOP). Each GOP is typically built starting with an I-FRAME, which is encoded independently to other frames, and P-Frames and B-frames which carry the motion differences. Longer chunks can adopt a larger GOP, thus greatly reducing the average chunk size. At the same time, longer video chunks have more problems when dealing with sudden network changes, as they need more time to complete the download and thus react and change the quality according to the current network performance.

In this paper we present an analytic study of the parameters to be taken into account when using a static SS for the playback, like the data rate of the connection in use, its latency, and the mobility of the user. We then define our proposal, where we monitor the connection stability in terms of throughput, and dynamically adapt to the network conditions by changing the SS. Our experimental results show that our algorithm is able to provide a smooth playback like the shorter SS do, and it is also able to increase the quality by downloading larger segments when the connection performance is stable. This comes from the fact that the video size for larger SS is smaller than the video size for a shorter SS, and therefore it can be downloaded using the bandwidth more efficiently.

The rest of this paper is structured as follows: in Section II we present and discuss related proposals from literature. Section III introduces and analyzes the tradeoff between fixed size segment selection and dynamic size segment selection, which also constitutes our proposal. Section IV provides a performance evaluation of the presented schemes, and Section VII discusses the results and concludes this paper.

II. RELATED WORK

Dynamic adaptive video streaming has gained a lot of interest recently, mostly thanks to its ability to adapt to varying

network conditions and possibly provide a better quality of experience (QoE) to the end users. Many studies show how an interruption in the video playback is detrimental for the users to continue the playback [4] [10].

It is worth to note the Pippi Longstocking Calculus [4], which studies the effect of video pauses on Youtube users QoE. The main result from the study is that the total number of pauses has a stronger impact on the QoE rather than the length of the pauses. Two of the examples shown in the paper reveal that a single 3 seconds pause reduces the QoE like 2 pauses of 1 second. Therefore, it is important to limit the total seconds of pauses, but also the number of them. The part of the MPEG-DASH player in charge of selecting the quality of the video segments to be played is called the rate adaptation algorithm. In literature it is possible to find several works on rate adaptation algorithms, which forecasts the network performance in order to select an appropriate quality. The study presented in [15] shows the segment quality selection according to different throughput estimators, two of which are the Last Segment Throughput and the Smoothed Throughput. The first one determines the throughput by computing it through the download of the last segment. However, it suffers from instantaneous variations which might give inaccurate estimations. This is partially solved by the second method, which computes an average over the last segments, to reduce the effect of sudden variations.

The BOLA proposal [14] monitors the buffer level, in order to perform a careful selection on the quality to be streamed. The algorithm achieves good performance, and is evaluated under different network conditions. Other works are presented in [6], where the authors focus on Content Distribution Networks [1] and propose algorithms that can detect network congestion and act accordingly. However, most of the works focus on rate adaptation algorithm that fixes a SS, and acts on varying the quality of the segment to be downloaded and possibly played. Finally, it is worth to mention the DASH-JS [11] player, which is the reference client offered by the DASH Industry forum, and is available online for anyone willing to use it or to improve it. The DASH Industry forum groups together important stakeholders such as Microsoft, Qualcomm and Akamai, to name few. Few works target the SS as a parameter to be adjusted according to the changing network conditions. One of them is [8], where the authors evaluate the effect of different SS on a video, in terms of buffer size, playback quality and number of requests made to the server. Concerning the latter, it is straightforward to note that a shorter SS eventually forces the client to make more GET request to get the video. At the same time, short SS keep the buffer full, thanks to a lower time needed to get the segment. This is a somehow similar conclusion as the one obtained in [9], where the authors study other interesting parameters such as the buffer overflow probability. Finally, it is worth to note the work in [12], where the authors analyze the Quality of Experience for the users when varying the SS. They place two concurrent users downloading the same video, but with different SS, and analyze the impact on the QoE

TABLE I
AVERAGE BITRATES FOR THE BIG BUCK BUNNY VIDEO [5]

SS length (s)	Minimum Quality (Mbps) (QI = 0)	Maximum Quality (Mbps) (QI = 19)
1	0.047	4.72
2	0.046	4.21
4	0.045	3.93
6	0.045	3.85
10	0.045	3.79
15	0.045	3.74

for both. MPEG-DASH has also been evaluated for vehicular environments [7], where the authors also present a comparison between different dynamic adaptive video streaming protocols, which also highlight the differences in the rate adaptation algorithms.

Compared to the previous work found in literature, we provide a deep analysis on the performance of fixed SS selections for MPEG-DASH videos. We also combine our findings to provide a SS adaptive algorithm, which based on the network conditions (both in terms of throughput and stability) adapts the SS to provide a higher quality to the end user, while keeping the interruptions low, comparable to that experienced when using the smallest possible SS.

III. SEGMENT SIZE SELECTION

In this section we discuss the factors that affect the video playback when using a fixed SS, and the benefits of a dynamic SS selection in terms of streamed video quality and reduced pause times. Videos that are streamed using MPEG-DASH are typically encoded in different SS and qualities. At the beginning of each segment, it is typically placed an MPEG I-FRAME, which contains the full image frame, followed by a series of P-FRAMES, which describe the changes in the video hence are smaller in sizes. This is mainly because each video segment should be self-contained, so when the quality is changed the video can continue to play without further data to be downloaded. This is also why for smaller SS the video size for the same length can be larger than longer SS, since to play the same content more I-FRAME are needed, and thus the total size of the packet for the same video length increases. Also, when the user skips a portion of the video, the playback should be able to start from the point the user skipped to, without the need to download previous video segments. If TCP is used as Transport Protocol, a new connection is instantiated for every packet to be downloaded. It is immediately evident how a higher number of HTTP GET leads to an increased overhead, due to the higher number of TCP connections; a problem happening more with shorter SS. However, this behavior can be mitigated by using HTTP persistent connections, where it is not needed to instantiate a new TCP connection for every segment, but only one at the beginning.

A. Fixed Segment Size

For our tests, we use the Big Buck Bunny video from [5], which is encoded in 6 different SS (i.e. 1, 2, 4, 6, 10 and 15 seconds) and with 20 different qualities. An overview of the

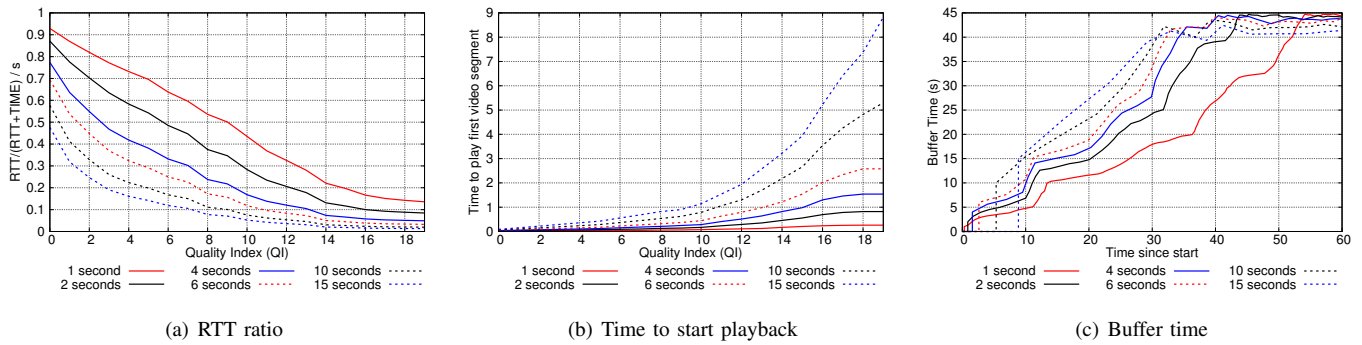


Fig. 1. Figure 1(a) shows the RTT ratio between the time needed to establish a connection and time needed to fully download the packet. Figure 1(b) shows the time needed to play the first segment of the video, while varying the QI. Finally, Figure 1(c) shows how the buffer size evolves taking into account different SS. All the charts refer to a 10 Mbps connection with a 20 ms connection latency.

different video sizes is given in Table I, highlighting the video size differences from larger SS compared to smaller SS.

In Figure 1(a) we plot the overhead of the different SS when varying the quality in play. To perform a fair analysis, we use the Quality Index (QI) rather than the video bitrate in play. This is simply because shorter SS would have a higher bitrate than larger SS for the same quality. The QI is simply the index used to represent the different qualities: in the dataset we used [5], the Big Buck Bunny video has 20 different video encodings, labeled from QI=0 to QI=19, with higher QI representing better quality videos. The total video time is about 10 minutes. A larger SS leads to a reduced overhead, as the time needed to download the segment is much greater compared to the time needed to initiate the connection, and also involves less connections for the same video length. On the inverse, when playing a lower quality, the time needed to handshake the connection is greater than the time needed to effectively download the segment. As stated before, this behavior is also influenced by the SS in play, as larger SS download bigger packets compared to smaller SS, and thus mitigate the overhead by lowering the total number of connections to instantiate. Faster connections suffer more from the latency, as the time needed to download the packet is significantly less compared to slower connections, in which the latency of the connection has a reduced impact. We also note that this behavior is partially mitigated by the rate adaptation algorithm, which selects the appropriate quality to be played according to the network bandwidth.

Figure 1(b) shows the time needed to start the video playback, depending on the quality. Again, large SS delay the time needed to start the video playback, since it is needed more time to download the first packet containing the initial video chunks to be played. Small SS have the benefit to download smaller packets, hence the time needed to begin the video playback is much reduced to less than 1 second for $SS = 1$ and $SS = 2$. This behavior can have an impact for short videos, where an initial pause of $\simeq 10$ seconds might represent a considerable time. It is also important to note that perceived bad connections lead to users quitting the playback of the video earlier [10]. However, the SS also affects the buffer, as

it can be seen from Figure 1(c), where we plot the buffer time according to different SS as the video plays. Larger SS have the benefit of needing less bandwidth compared to smaller SS at the same quality, and thus they fill the buffer more quickly. For this experiment we set the maximum buffer time to 45 seconds, and we see that when it is full, larger SS tend to have a lower average buffer time. This is simply explained by the fact that a new video segment is downloaded every time the buffer falls below 45 seconds. At that time, a small SS takes less time to download the packet and thus to refill the buffer, while a larger SS takes more time hence the buffer empties for more time.

It is straightforward to understand how small SS should be preferred when the buffer level is low, to limit the pause times, and longer SS should instead be used to optimize the bandwidth usage. However, it is also important to keep in mind the effects of a variable connection, in which the throughput might not be always the same, and thus longer SS downloaded at an high quality might become stuck by sudden dropping throughput. Here, smaller SS would allow for a fast recovery action by reducing the quality immediately.

B. Dynamic Segment Size

In this section we describe our proposal to dynamically select the SS based on the network performance. A smaller SS has the benefit of better accommodating throughput changes, at the cost of an increased bandwidth for the same video quality. We design our scheme by leveraging the stability of the connection and its average throughput. Thus, at the beginning of every playback, we chose $SS = 1$, since we do not know the performance of the network, and also to start the playback earlier than having longer SS, as Figure 1(b) shows. At first, we define a vector $\Omega = \{\omega_1, \dots, \omega_N\}$ containing the list of possible SS for the video in play, ordered from the shortest to the longest. We note that the time between consecutive SS might be different for each couple, since the SS length selection scheme is dependent on the choices of the content provider. There is also a storage issue in maintaining several SS, since for each one the content provider has to provide different encoding qualities, thus raising the storage needed to

keep the same video. Therefore, we design our scheme to be independent by the number and the values of the offered SS. We also define a vector $\Lambda = \{\lambda_1, \dots, \lambda_M\}$, which contains the last M throughputs experienced, from the oldest one (i.e. λ_1) to the most recent one (i.e. λ_M). Therefore, the average throughput is easily computed as

$$\mu = \frac{\sum_{i=1}^M \lambda_i}{M} \quad (1)$$

The term μ gives us an indication on the average streaming quality that the connection could offer. However, knowing the average throughput μ is not enough, as also the stability of the connection plays an important role in selecting the most appropriate video segment to be downloaded. If the connection is unstable, it is preferable to choose a short SS rather than a longer one, to better accommodate quality changes. On the inverse, a stable connection should lead to a larger SS, to download longer segments which at the same video quality have a smaller video size. To better cope with the varying environment, and to react faster after sudden throughput variations, we have to prioritize recent throughputs compared to older ones. Therefore we define σ_{weight}^2 as:

$$\sigma_{weight}^2 = \sum_{i=1}^M \frac{i}{M} |\lambda_i - \mu| \quad (2)$$

Having computed μ and σ_{weight}^2 , we define a probability $p \in [0, 1]$, which estimates the willingness of changing SS:

$$p = \frac{\mu}{\mu + \sigma_{weight}^2} \quad (3)$$

For stable connections $p \simeq 1$, while an unstable connection will decrease p . However, if the throughput is much higher than its variance, p will be still kept close to 1. We use p to estimate the possibility to increase or decrease SS as follows:

$$\tau = (1 - p) \cdot \Omega[\max(0, SS - 1)] \quad (4)$$

$$\theta = p \cdot \Omega[\min(N, SS + 1)] \quad (5)$$

Basically, τ defines the willingness to decrease the SS, while on the inverse θ defines the willingness to increase it. As we already stated, a $p \simeq 1$ will result in a τ close to 0 and θ close to the next SS bigger than the one we are using. On the inverse, when $p \simeq 0$, which happens under unstable connections, τ will be close to the previous SS smaller than the one we are using, and clearly θ will be close to 0. We compute the new SS as:

$$SS' = \operatorname{argmin}(\Omega[i] - \tau + \theta) \quad (6)$$

in which we pick the closest SS to the one computed by $\tau + \theta$. We note that the SS in play could also remain the same as the previous packet downloaded. With unstable connections, we switch back to shorter SS, to accommodate possible quality changes if the throughput varies. Stable networks will increase the SS to reduce the number of connections and possibly utilize more efficiently the bandwidth. Our proposal does not focus on the actual value of the bandwidth, as rate adaptation algorithms do. We focus on its stability, thus slow but stable

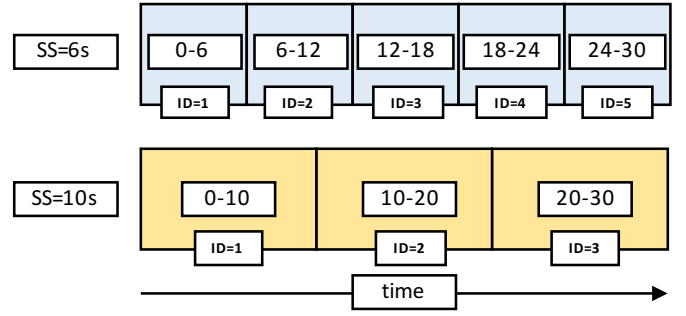


Fig. 2. The overlapping problem. A perfect switch (i.e. with no overlapping) could happen only after SS_5 . In all the other case, switching to SS_{10} would cause the client to download again parts of the video already in the buffer.

connections could still be used to download longer SS. In fact, our dynamic SS selection algorithm runs after the quality has been selected by the rate adaptation algorithm.

However, there is another issue to discuss when changing the SS, which is related to the seconds of video already downloaded, which might be overlapped when changing the SS. For instance, let's assume that a video is encoded with $SS = 6$ and $SS = 10$, so $\Omega = \{6, 10\}$. Assume that the SS chosen is 6, and that we successfully download segments SS_1^6, SS_2^6, SS_3^6 . Therefore, the buffer fills up to 18 seconds. At this time, the connection stabilizes, and the algorithm would switch to $SS = 10$. However, the segment that has to be downloaded is SS_{10}^{10} , which has 8 seconds overlapped with the buffer, since we already have downloaded them before, and therefore only 2 seconds of it will actually be useful for the playback. This behavior is inefficient for the bandwidth, hence it is important to understand when it is beneficial to switch rather than just staying with the current SS, although it is not the one foreseen by the dynamic algorithm, and postpone the switching. To evaluate the switching, we compute the segment ϕ to be downloaded as $\phi = \left\lfloor \frac{T+B}{SS} \right\rfloor$ where T is the actual playback time, and B is the size of the buffer. We then compute the overlapping δ as:

$$\delta = \frac{\sum_{i=T}^{T+B} B(i)}{SS} \quad (7)$$

where $B(i)$ is 1 if the video at position i is already in the buffer B , and 0 otherwise. Should it has been already downloaded a huge portion of video compared to the new SS, δ will be close to 1. With probability δ we keep the SS stable, and with probability $(1 - \delta)$ we change it to the new one computed through Equation 6 (i.e. $SS = SS'$). Higher δ represent high overlapping, which result in downloading huge portions of video another time even though that video frames are already in the buffer. On the inverse, when $\delta \simeq 0$, the overlapping is small, and thus we can change SS, if the algorithm chooses to, avoiding to download the video frame again.

IV. PERFORMANCE EVALUATION

In this section we evaluate the dynamic SS selection against fixed SS selections. Like in the previous section, we use as

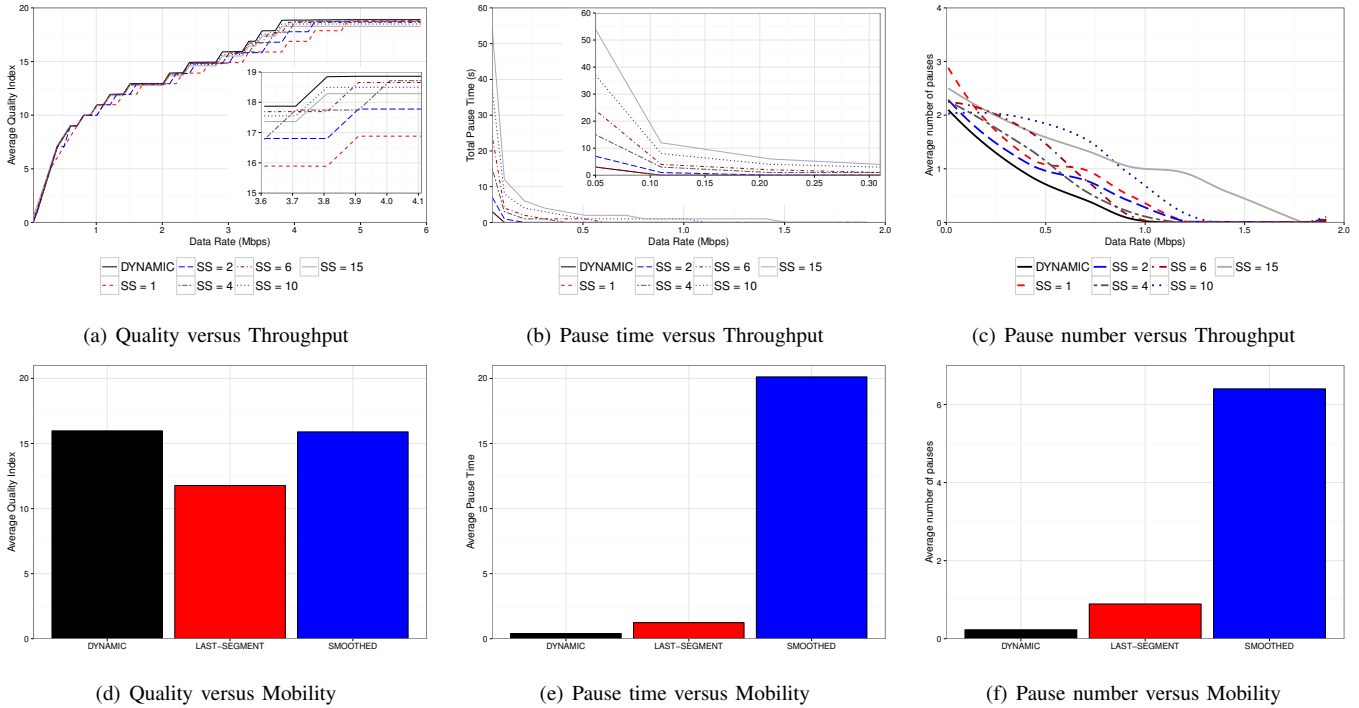


Fig. 3. Figures 3(a)-3(c) show the performance of the different SS selection versus the offered throughput. Figures 3(d)-3(f) is the mobility analysis.

reference video Big Buck Bunny from [5]. Table I reports the two extreme QI for each SS of the video, thus highlighting the differences in terms of video size, as the maximum quality ranges from 4.7 Mbps for $SS = 1$ to 3.7 Mbps when $SS = 15$, which constitutes a difference of about 1 Mbps in the bandwidth requirements for the video playback. To perform our analysis, we implemented an Android Application using the well known Exoplayer library¹. We evaluate three different proposals: LAST-SEGMENT is a fixed SS selection, in which the rate adaptation selects the highest quality with an average size lower than the last throughput, SMOOTHED which performs a similar choice but looking at a longer throughput history, and DYNAMIC which uses WISER as rate adaptation algorithm, but performs the dynamic SS selection as detailed in Section III-B [15]. We perform two different analysis: the first is a static scenario, in which we consider a single client streaming a video, presented in Section V. We bind the mobile video client to a WIFI AP in which we constrain the outbound connection between 50 kbps and 6 Mbps. For the second evaluation, presented in Section VI, we walk outside in the University of Bologna campus, which offers a WIFI network available for students and researchers. We disabled the cellular data connection, so that only WIFI can be used.

V. STATIC ANALYSIS

As we can see from Figure 3(a), our proposal achieves better results than the others in term of QI in play, since it better adapts to the bandwidth offered by the network. At the beginning of the playback, our proposal selects the lowest

possible SS (i.e. $SS = 1$). When it understands the throughput and the stability of the connection, it switches to longer SS to better utilize the offered throughput. The lower quality offered by fixed long SS comes from a lower quality chosen at the beginning of the playback. There, shorter SS make the rate adaptation algorithm able to understand sooner that with the offered throughput the client is able to play a higher video quality, and thus switch to it after less time compared to longer SS, which instead have to wait for the initial segments to be downloaded, and after that switch to an higher quality. This is clearly a positive aspect considered the average lower video length played by user in presence of bad connections [10].

Figure 3(b) highlights the fact that shorter SS experience less pause times compared to longer ones. This is clearly dependent on the variance of the connectivity, as shorter SS have more opportunities to change the quality when the connection drops, while longer SS have to wait more time when a packet is downloaded, and thus the quality change happens later. We note that our proposal achieves better performance as the short SS, by using a short SS to start the playback sooner, and increasing it later to leverage the lower video size for longer SS. The reduced pause time comes from a reduced bandwidth needed when longer SS are selected. However, fixed longer SS suffer from an initial pause, as more time is needed to play the video, like we have already shown in Figures 1(b)-1(c).

As [4] states, the total number of pauses has a stronger effect on the QoE rather than the total length. Therefore, in Figure 3(c) we evaluate the number of pauses in each playback, averaged over the total number of runs we performed. Again, our proposal performs better than the others, by using a shorter

¹<https://github.com/google/ExoPlayer>

SS to fill rapidly the buffer, then switching to a longer SS to utilize more efficiently the throughput. The stability of the connection enables a switch to a longer SS even with low throughput, to utilize the connection more efficiently.

VI. MOBILE ANALYSIS

The next analysis is to account for mobility effects on the selection of the SS. Figures 3(d)-3(f) picture the results of our test, for the same metrics evaluated in the static scenario. For the fixed SS selection algorithms, we have chosen $SS = 10$.

Figure 3(d) is the QI experienced by the different algorithms evaluated during our tests. It can be seen that there are some algorithms which perform better than others, like the DYNAMIC and the WISER, while ADAPTIVE perform worse. This is driven by the fact that ADAPTIVE monitors only the last throughput instead of a longer history, and therefore quickly drops the quality when experiencing a lower throughput, therefore not fully exploiting the buffer that it might have already downloaded. WISER instead monitors a longer history, and therefore maintains a higher quality for a longer time before dropping it. DYNAMIC slightly improves this, by also selecting longer SS when the throughput stabilizes, thus needing less bandwidth to download similar video qualities.

Figure 3(e) and Figure 3(f) show a similar behavior, and highlight the fact that although WISER achieves comparable performance to DYNAMIC concerning the QI, this is not the same when evaluating the pauses. Again, this happens because a static SS selection is less reactive to accommodate quality changes rather than a dynamic one, which can exploit longer SS when the throughput is stable, and move to shorter ones when it is uncertain the ideal selection that has to be done.

VII. CONCLUSION

In this work we have presented a study on the tradeoffs introduced by different SS in dynamically adaptive video streaming. At first, we have shown the differences in terms of download times, playback times, and buffer behavior by taking into account different SS. The rationale is that shorter SS can favor a smooth video playback with less interruptions, at the cost of a possible larger average size due to a smaller GOP, which is also the reason why with stable connections it is better to select a larger SS. The mobility scenario however highlighted the fact that this behavior might does not need to be always true, as longer SS can benefit from a reduced video size compared to shorter SS. It is worth to note that video content provider will however provide a reduced set of SS to choose from, to occupy less disk storage. We then presented our proposed dynamic SS adaptation algorithm, which monitors the connection performance and adapts accordingly, by switching to shorter SS when the connection is unstable, and increasing the SS when the connection allows. Performance evaluation clearly shown the benefits of our proposal against classical fixed SS selection schemes in terms of reduced pause times and events, and increase quality.

Future work on this topic include adding a preference from the user, which will be able to choose either to improve the

quality at the expense of adding more pauses, or reduce the pauses by lowering the quality. We also foresee to implement and evaluate additional rate adaptation algorithms and evaluate the joint benefits of both a dynamic SS selection as well as the dynamic quality selection. Mostly interesting here would be the difference between buffer based, throughput based, and mixed algorithms [3]. We plan to evaluate multi homed devices, by exploiting multiple connections to download the same video with different SS, to gain the benefits of both, or possibly playing different videos using different connections. We also plan to study the effect of persistent versus non-persistent HTTP connections on the video playback, which will reduce the impact of the latency the connection, which has a strong impact particularly when considering low throughput connections where the RTT plays a key role.

REFERENCES

- [1] Miran Abdullah. A Novel CDN Testbed for Fast Deploying HTTP Adaptive Video Streaming. In *Proceedings of the 9th EAI International Conference on Mobile Multimedia Communications*. ACM, 2016.
- [2] The Cisco. Cisco Visual Networking Index : Global Mobile Data Traffic Forecast Update , 2014 - 2019. 2011(4):2010–2015, 2011.
- [3] Florin Dobrian, Asad Awan, Dilip Joseph, Aditya Ganjam, Jibin Zhan, Vyas Sekar, Ion Stoica, and Hui Zhang. Understanding the impact of video quality on user engagement. *Communications of the ACM*, 56(3):91, mar 2013.
- [4] Tobias Hossfeld, Dominik Strohmeier, Alexander Raake, and Raimund Schatz. Pippi Longstocking calculus for temporal stimuli pattern on YouTube QoE. In *Proceedings of the 5th Workshop on Mobile Video - MoVid '13*, page 37, New York, New York, USA, 2013. ACM Press.
- [5] Stefan Lederer, Christopher Mueller, Christian Timmerer, Cyril Concolato, Jean Le Feuvre, and Karel Fliegel. Distributed DASH dataset. In *Proceedings of the 4th ACM Multimedia Systems Conference on - MMSys '13*, pages 131–135, New York, USA, 2013. ACM Press.
- [6] Chenghao Liu, Imad Bouazizi, Miska M. Hannuksela, and Moncef Gabbouj. Rate adaptation for dynamic adaptive streaming over HTTP in content distribution network. *Signal Processing: Image Communication*, 27(4):288–311, apr 2012.
- [7] Christopher Müller, Stefan Lederer, and Christian Timmerer. An evaluation of dynamic adaptive streaming over HTTP in vehicular environments. *Proceedings of the 4th Workshop on Mobile Video - MoVid '12*, page 37, 2012.
- [8] Dung M. Nguyen, Long B. Tran, Hung T. Le, Nam Pham Ngoc, and Truong Cong Thang. An evaluation of segment duration effects in HTTP adaptive streaming over mobile networks. In 2nd National Foundation for Science and Technology Development Conference on Information and Computer Science (NICS), pages 248–253. IEEE, sep 2015.
- [9] Ognenoski O., Martini M.G., Amon P. Segment-based teletraffic model for MPEG-DASH. In *2013 IEEE 15th International Workshop on Multimedia Signal Processing (MMSP)*, pages 333–337. IEEE, sep 2013.
- [10] Louis Plissonneau and Ernst Biersack. A longitudinal view of HTTP video streaming performance. *Proceedings of the 3rd Multimedia Systems Conference on - MMSys '12*, page 203, 2012.
- [11] B. Rained, Stefan Lederer, C. Muller, and Christian Timmerer. A seamless Web integration of adaptive HTTP streaming. In *Proceedings of European Signal Processing Conference*, 2012.
- [12] A. Sideris, E. Markakis, N. Zotos, E. Pallis, and C. Skianis. MPEG-DASH users' QoE: The segment duration effect. In *2015 Seventh International Workshop on Quality of Multimedia Experience (QoMEX)*, pages 1–6. IEEE, may 2015.
- [13] Iraj Sodagar. The MPEG-DASH Standard for Multimedia Streaming Over the Internet. *IEEE Multimedia*, 18(4):62–67, apr 2011.
- [14] Kevin Spiteri, Rahul Urgaonkar, and Ramesh K. Sitaraman. BOLA: Near-optimal bitrate adaptation for online videos. In *IEEE INFOCOM 2016 - The 35th Annual IEEE International Conference on Computer Communications*, volume 2016-July, pages 1–9. IEEE, apr 2016.
- [15] Truong Thang, Quang-Dung Ho, Jung Kang, and Anh Pham. Adaptive streaming of audiovisual content using MPEG DASH. *IEEE Transactions on Consumer Electronics*, 58(1):78–85, feb 2012.